

ABAC, permissions boundaries and least privilege at scale

One policy for a thousand people – and the escalation path that opens if you get the tags wrong.

Once tags decide access, **whoever can set a tag can set permissions.**

RBAC, or ABAC?

RBAC

Permissions per role

- A policy per job function
- New project means new policies
- Easy to read: what does this role do?
- Policy count grows with the business

ABAC

Permissions per attribute

- One policy comparing tags
- New project means new tag values
- Easy to scale, harder to audit
- Policy count stays flat

THE TIEBREAKER

'Fourteen near-identical policies differing in one value' is the ABAC signal. So is 'we add a new team every month'.

The pattern, in three parts

- ◆ **Tag the principal.** A tag on the role, or a session tag from your identity provider – the module 21 and 22 mechanism.
- ◆ **Tag the resource.** Consistently, and enforced at creation, or the whole scheme has holes.
- ◆ **Write one policy** comparing **aws:PrincipalTag/team** against **aws:ResourceTag/team**.
- ◆ **aws:RequestTag** controls what tags may be applied in a request. **aws:TagKeys** controls which keys may appear at all.
- ◆ Tag and untag requests can carry several tags, so **aws:TagKeys** requires **ForAllValues** or **ForAnyValue** – with the module 20 empty-set caveat.

ON THE EXAM

Four condition keys: PrincipalTag on the caller, ResourceTag on the target, RequestTag on what is being set, TagKeys on which keys.

Three ways ABAC fails

- ◆ **Missing tags.** An untagged resource falls outside the scheme – a Deny on `aws:ResourceTag/Environment` does nothing without that tag.
- ◆ **Inconsistent tags.** Production, production, prod and PROD are four values. Mismatches cause silent denials or silent grants.
- ◆ **Tag manipulation.** The dangerous one – retag a resource to match your own attributes.
- ◆ Missing tags is the module 20 lesson: to **require** a tag you need **Null**.
- ◆ Inconsistent tags is why **tag policies** exist in AWS Organizations.

ON THE EXAM

The first two cause outages. The third causes privilege escalation.

Protecting the tags

- ◆ **Deny modification of the tag keys your policies depend on**, except for a small set of principals. This is the control most designs omit.
- ◆ Scope tagging permission with **aws:RequestTag** so a principal may set only certain values, and **aws:TagKeys** so only certain keys.
- ◆ Watch **tag-on-create**. Many services let you tag while creating a resource, which quietly grants tagging permission to anyone who may create.
- ◆ An **SCP** is the right altitude for this – it caps every principal in the account, including local administrators.
- ◆ In KMS, changing a tag **or an alias** can grant or remove access to a key. AWS says so explicitly.

ON THE EXAM

If your ABAC design does not say who may write the deciding tag, it is incomplete.

ABAC does not work everywhere

- ◆ AWS documents per-service ABAC support as **Yes, Partial or No**. Support is uneven and the condition keys are not consistent.
- ◆ **S3 uses its own keys** – **s3:ExistingObjectTag** for objects, not **aws:ResourceTag**. Bucket-level actions cannot be controlled by tag at all.
- ◆ **You cannot restrict iam:PassRole by tags on the role**. Tempting, and it does not work – scope **PassRole** by ARN, as module 21 said.
- ◆ EC2 offers **ec2:ResourceTag** as a synonym for the global key. Some services support neither.
- ◆ 2026: ABAC extended to **S3 Tables and S3 Access Points**, so coverage is improving.

ON THE EXAM

Test an ABAC policy before relying on it, especially a Deny. A condition key that does not exist for that service never matches.

Permissions boundaries for safe delegation

- ◆ The problem: a team must create roles for its own workloads, and creating roles means granting permissions.
- ◆ Grant **iam:CreateRole** and **iam:PutRolePolicy** with a condition requiring **iam:PermissionsBoundary** to equal your boundary ARN.
- ◆ Now they create any role they like, and every one is capped by your boundary.
- ◆ **Also deny removing or changing the boundary**, or they remove the cap in a second call.
- ◆ And deny editing the boundary policy itself. A team that can edit the ceiling has none.

ON THE EXAM

Three denies: cannot create without it, cannot remove it, cannot edit it.

Least privilege at scale

- ◆ Start from a managed policy, then narrow. A working over-broad policy beats a broken narrow one.
- ◆ **Last accessed data** shows which services and actions were actually used. 90 days idle is a candidate.
- ◆ **Access Analyzer generates policies from CloudTrail activity** – least privilege based on what happened.
- ◆ **Unused access findings** surface roles and permissions nobody exercises. Module 25.
- ◆ Least privilege is a direction, not a state. Every technique here reads history.

ON THE EXAM

None of these knows what the workload will legitimately need next quarter.

A company uses ABAC: developers carry a team tag from their identity provider, EC2 instances carry a matching team tag, and one policy allows `ec2:*` where `aws:PrincipalTag/team` equals `aws:ResourceTag/team`.

Developers also hold `ec2:CreateTags` on all resources, because they tag their own instances at launch. A penetration test shows a developer on the web team adding `team=web` to a production database's EC2 instance, then stopping it.

What should the company implement?

- ☐ A Restrict tagging with `aws:RequestTag` and `aws:TagKeys`, and deny changes to the team tag on existing resources
- ☐ B Move the team tag from the principal to a Cognito group claim
- ☐ C Replace ABAC with one IAM policy per team, listing instance ARNs
- ☐ D Add a Config rule that detects and reverts unauthorised tag changes

◆ PAUSE AND DECIDE

What should the company implement?

- A

 Restrict tagging with `aws:RequestTag` and `aws:TagKeys`, and deny changes to the team tag
Correct. The tag is the access decision, so writing it must be as controlled as the access itself. This closes the escalation without abandoning ABAC.
- B

 Move the team tag to a Cognito group claim
Changes where the principal's attribute comes from. The escalation was on the resource side – the developer retagged an instance, not themselves.
- C

 Replace ABAC with a policy per team
Removes the escalation by removing the scheme, and returns the company to the policy sprawl ABAC was adopted to solve. It also does not stop tag-based cost or automation drift.
- D

 Config rule to detect and revert
Detective and corrective. There is a window between retagging and reversion, and the developer only needs one API call inside it.

THE SIGNAL

The tag was the access decision, and writing it was unguarded. ABAC without tag governance is only half a design.

A platform team wants to limit `iam:PassRole`. Their idea is to tag application roles with `Purpose=workload` and administrative roles with `Purpose=admin`, then allow `iam:PassRole` only where `aws:ResourceTag/Purpose` equals `workload`.

In testing, the condition never matches. Every `PassRole` attempt is denied, including for correctly tagged workload roles.

Why, and what should they do instead?

- ☐ A Tags on roles propagate asynchronously; wait for consistency and retry
- ☐ B The condition must use `iam:ResourceTag` rather than the global key
- ☐ C You cannot restrict `iam:PassRole` using tags on the role; scope it by role ARN and `iam:PassedToService`
- ☐ D `PassRole` requires the tag on the passing principal, not on the role

◆ PAUSE AND DECIDE

Why, and what should they do instead?

- A

 Wait for tag propagation
Consistent, permanent failure across every correctly tagged role is not eventual consistency. Waiting changes nothing.
- B

 Use iam:ResourceTag instead
The right instinct – service-specific keys do exist, as EC2 shows. IAM does not offer one for this, and no key name makes PassRole tag-aware.
- C

 PassRole cannot be restricted by tags on the role; use ARNs and PassedToService
Correct. This is a documented ABAC gap. The workable control is the module 21 one: enumerate permitted role ARNs and add iam:PassedToService.
- D

 Tag the passing principal instead
Would restrict who may pass anything, not which roles may be passed. The requirement is about the roles, so this solves a different problem.

THE SIGNAL

ABAC support is per service and per action. A condition key that does not apply never matches – it does not error.

An analytics team applies its ABAC model to S3. Objects are tagged Project=alpha or Project=beta, and analysts carry a matching principal tag. The policy allows s3:GetObject where aws:ResourceTag/Project equals aws:PrincipalTag/Project.

The policy works perfectly for EC2 and Secrets Manager resources under the same model. For S3 objects it never grants access, and no error indicates why.

What is wrong?

- ☐ A S3 object tags must be applied at upload; tags added later are ignored by IAM
- ☐ B S3 uses s3:ExistingObjectTag for object tags, not aws:ResourceTag
- ☐ C The bucket policy must also grant s3:GetObjectTagging for the condition to evaluate
- ☐ D Principal tags are not available for S3 requests; use a session policy instead

◆ PAUSE AND DECIDE

What is wrong?

- A

 Tags added after upload are ignored
Invented. Object tags can be changed at any time and are evaluated when the request is made, whenever they were set.
- B

 S3 uses s3:ExistingObjectTag, not aws:ResourceTag
Correct. S3's tag condition keys are service-specific. The global key is simply absent from an S3 object request, so the condition never matches.
- C

 Grant s3:GetObjectTagging too
That permission governs reading tags via the API. Policy evaluation reads the object's tags directly and needs no extra grant.
- D

 Principal tags unavailable for S3
aws:PrincipalTag is a global key present on every request regardless of service. The problem is on the resource side of the comparison.

THE SIGNAL

The same model working for EC2 and Secrets Manager and failing only for S3 is the tell. It is the service, not the design.

A platform team must let 12 application teams create their own IAM roles for Lambda functions, without being able to create a role that exceeds a defined ceiling. Waiting for the platform team to create every role has become a delivery bottleneck.

The proposal is to grant each team `iam:CreateRole` and `iam:PutRolePolicy`, with a condition requiring `iam:PermissionsBoundary` to equal the platform team's boundary ARN.

What else is required for this to be safe?

- ☐ A Nothing further; the condition on `iam:PermissionsBoundary` is sufficient
- ☐ B Also grant `iam:AttachRolePolicy` so teams can use AWS managed policies
- ☐ C An SCP denying all IAM actions in the application accounts
- ☐ D Deny removing or changing the boundary on a role, and deny editing the boundary policy itself

◆ PAUSE AND DECIDE

What else is required for this to be safe?

- A

 The condition alone is sufficient
Covers creation and nothing after it. A team that can create a bounded role and then delete the boundary has created an unbounded one in two steps.
- B

 Also grant iam:AttachRolePolicy
Convenient and it widens the grant rather than securing it. Attached managed policies are still capped by the boundary, but this answers a different question.
- C

 SCP denying all IAM actions
Would block the delegation entirely, including the CreateRole the whole design depends on. It solves the bottleneck by making it permanent.
- D

 Deny removing or changing the boundary, and deny editing the boundary policy
Correct. Three controls make this work: cannot create without the boundary, cannot remove it afterwards, cannot edit the ceiling itself.

THE SIGNAL

A delegation control has to hold at creation and afterwards. Ask what the delegate can do on their second API call.

Five things to carry forward

- ◆ **Whoever can set a tag can set permissions.** An ABAC design without tag governance is half a design.
- ◆ **Missing tags fall outside the scheme.** To require a tag you need Null, not a comparison.
- ◆ **ABAC support is per service and inconsistent.** S3 uses s3:ExistingObjectTag; PassRole cannot be restricted by tag at all.
- ◆ **Delegation needs three denials** – cannot create without the boundary, cannot remove it, cannot edit it.
- ◆ **A condition key that does not apply never matches, and never errors.** Consistent silent denial is the signature.

ON THE EXAM

Next: why was this denied – Access Analyzer, the simulator and CloudTrail.